validated the token. It was even possible to change it to the value `none`, which instructed the JWT library to not validate the signature at all! These kinds of security flaws have led some people to argue that JWTs are inherently insecure due to the ease with which they can be misused, and the poor security of some of the standard algorithms.

PASETO: an alternative to JOSE

The error-prone nature of the standards has led to the development of alternative formats, intended to be used for many of the same purposes as JOSE but with fewer tricky implementation details and opportunities for misuse. One example is PASETO (https://paseto.io), which provides either symmetric authenticated encryption or public key signed JSON objects, covering many of the same use-cases as the JOSE and JWT standards. The main difference from JOSE is that PASETO only allows a developer to specify a format version. Each version uses a fixed set of cryptographic algorithms rather than allowing a wide choice of algorithms: version 1 requires widely implemented algorithms such as AES and RSA, while version 2 requires more modern but less widely implemented algorithms such as Ed25519. This gives an attacker much less scope to confuse the implementation and the chosen algorithms have few known weaknesses.

I'll let you come to your own conclusions about whether to use JWTs. In this chapter you'll see how to implement some of the features of JWTs from scratch, so you can decide if the extra complexity is worth it. There are many cases in which JWTs cannot be avoided, so I'll point out security best practices and gotchas so that you can use them safely.

6.2.1 The standard JWT claims

One of the most useful parts of the JWT specification is the standard set of JSON object properties defined to hold claims about a subject, known as a *Claims Set*. You've